P3292R0R0
Provenance and Concurrency

Published Proposal, 2024-05-13

This version:
http://wg21.link/P3292R0

Author:
David Goldblatt

Audience:
SG1

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Table of Contents

§ 1. Introduction

When adapting lifetime rules to a concurrent C++, the committee added a sentence to `[basic.life]`: "In this subclause, 'before' and 'after' refer to the 'happens before' relation". In single-threaded C++, this is considerably simpler: given any two statements A and B, either A happens before B or B happens before A. In concurrent C++, though, the world has changed; now, A can happen before B, B can happen before A, or *neither of these things can be true*. It's not clear what the rules are in that third case.

As an example of where this gets tricky: After a pointer is passed to a deallocation function, all other pointers to the object become indeterminate values. But if "after" here means "any statement that the deallocation happens before", then the rule is much too weak (pointers in other threads remain valid). If it means "any statement that does not happen before the deallocation", then it disallows implementing various important concurrent algorithms. We're looking at the subtle issues this uncovers around the end of an object's lifetime in the pointer zap paper.

But there is another set of problems that a mix of academics and compiler engineers have been looking at, surrounding issues near the *beginning* of an object's lifetime (either before an object has been created, or before it has escaped to another thread). These problems seem to have the shape "analysis of memory dependencies says that two operations are independent and can be reordered, when in fact counterintuitive aspects of concurrency mean that they cannot":

- Synchronization in some opaque extern function call can affect memory that it doesn't access and whose address it may not even know.
- Load buffering can send values back in time, so they may have escaped to points sequenced before where they escape, or even the start of their lifetime; `void *p = new int` may alias other in-scope pointers without any UB.

The issue is that changing the compilers to strictly match the memory model disallows some useful and intuitive optimizations, and changing the memory model to strictly match the compilers results in some unworkably unfriendly semantics.

We'll look at two miscompilations, one I think the compilers are morally right about and one I think they're morally wrong about. I'll suggest a couple of directions for a fix; the least bad fix I can think of uses provenance rules to try to retroactively bless some of compilers' current behavior.

§ 2. The miscompilations

§ 2.1. Early Escape

Consider the following small snippet of code:

```
void some_extern_func(int);

int* f(int* q) {
    int* p = new int;
    *p = 123; // A
    *q = 456; // B
    some_extern_func(*p); // C. Can we rewrite this to `some_extern_func(12
    return p;
}
```

Clang and GCC both miscompile it at `-O1` and higher. They will constant-propagate the value stored into `*p` at `A` into the call at `C`, rewriting C into `some_extern_func(123)`. (Godbolt link)

This is not correct, because (surprisingly) it's possible for `p` and `q` to be equal without violating any lifetime rules, so that the store at `B` overwrites the one at `A`.

The reason that compilers make this mistake is that they employ reasoning like the following: "we see `*p`'s lifetime begin, and so any pointer value that was obtained before that point must point to some other object (or else violate provenance / 'pointer zap' rules).".

This belief is correct for single-threaded code, but turns out not to be correct in the presence of concurrency. The reason is that we can use load-buffering to send pointers to objects back in time to before their creation:

```
int* f(int* q) {
  int* p = new int;
  *p = 123; // A
  *q = 456; // B
```

```
  some_extern_func(*p); // C. Can we rewrite this to `some_extern_func(123)`?
  return p;
}

int dummy;
std::atomic<int *> ptr_to_int_1{&dummy};
std::atomic<int *> ptr_to_int_2{&dummy};

void t1() {
  int* q = ptr_to_int_1.load(relaxed);
  int* p = f(q);
  // Can get moved before the load of ptr_to_int_1
  ptr_to_int_2.store(p, relaxed);
}

void t2() {
  // Launders t1's store to send it back to t1.
  int* p = ptr_to_int_2.load(relaxed);
  ptr_to_int_1.store(p, relaxed);
}
```

At runtime, T1's CPU can reorder the store to `ptr_to_int_2` in front of the rest of `t1`, then T2 can read that stored value and copy it to `ptr_to_int_1`. Then `t1` can read it, passing it into `f`, so that `p` and `q` have the same value. Note that we don't violate any lifetime rules here – the start of the lifetime of the allocated `int` happens before any access of it. This program has defined semantics that the compiled code sometimes does not obey. Even though the user is manipulating pointers to objects before those objects' lifetime begins, they're not *dereferencing* those pointers, which is the thing that's not allowed.

Even though the compiler doesn't follow the letter of the law here, I think it's morally correct. Insofar as we have rules of engagement between the concurrent and single-threaded semantics, one of them is "SG1 shouldn't get in the way of 'normal' optimizations that aren't too aggressive and aren't in code that says `atomic`". Assuming that new storage doesn't alias anything in scope at the start of its lifetime is pretty reasonable, and it only turns out to be false because of strange concurrency patterns concocted to demonstrate compiler bugs.

§ 2.2. Store before fence

Consider the following snippet of code:

```
void some_extern_function();

int* f() {
  int *p = new int;
  *p = 123; // A
  some_extern_function(); // B
  return p;
}
```

Can the optimizer swap the store and the function call, and rewrite this into something that looks like the following?

```
int* f() {
  int *p = new int;
  some_extern_function(); // B
  *p = 123; // A
  return p;
}
```

Clang will sometimes, incorrectly, answer "yes". Thankfully, it won't actually do this transformation for small snippets like this one. But, due to a bug in the loop analysis passes, it sometimes will reorder the store and function call when some extra code is added. But these details are sort of extraneous -- the question of whether or not the compiler *will* reorder these is different than whether or not it *should be allowed to*.

The reason why this is incorrect is straightforward: if we fill in `some_extern_function()` with a release fence and add some callers, we can break simple message passing with fences:

```
atomic<int*> atomic_int_ptr;

void some_extern_function() {
  atomic_thread_fence(memory_order_release);
}

int* f() {
  int *p = new int;
```

```
  some_extern_function(); // B
  *p = 123; // A
  return p;
}

void message_pass_f() {
  int* p = f();
  atomic_int_ptr.store(p, relaxed);
}

void receive_message() {
  int* p = atomic_int_ptr.load(relaxed);
  atomic_thread_fence(acquire);
  if (p != nullptr) {
    assert(*p == 123);
  }
}
```

I think that, in this case, all reasonable people will agree that the memory model is working as intended and compilers should not be allowed to do the reordering. A compiler that does this transformation is buggy.

But: such a compiler could be forgiven its mistake. A similar sort of reasoning as above seems like it ought to apply: "`p`'s memory is private to me at the time of its allocation, so I can reorder it at will up until it escapes so long as it matches single-threaded semantics". Just as above, this reasoning is correct for single-threaded programs. Synchronization operations are weird: they can have effects on memory whose address they don't have access to.

§ 3. Provenance rules as the direction for a fix

When we're trying to describe the memory model, we're balancing the concerns of two groups:

- For compiler-writers: we need a simple rule that's quickly and locally (i.e. not requiring complex interprocedural or whole-program analysis) checkable rule for memory-dependence analyses to use. It should allow aggressive reordering in "reasonable" code, while staying correct with respect to some mathematical model.
- For memory-modelers: we need rules that are easy for users to reason about (insofar as the underlying hardware models permit) and allow reasonable concurrent code to stay well-defined in theory as well as in practice.

I think we can integrate provenance rules into the memory model to make some of this work.

The rule I want for compiler-writers is that we allow more or less all reorderings they currently do (that are consistent with single-threaded semantics), except across extern function calls that do release operations (this is the rule that LLVM already mostly tries to follow; the fact that it doesn't in some cases is an error, not a design choice). This might seem like it violates the "locally checkable" rule -- after all, we may not know at a call site to some extern function if that function has a release operation in it. But recent advances in LTO make this a more reasonable thing to assume. For instance, modern clang can do bottom-up whole-program call graph analysis without loading the whole program IR into memory.

The rule I want for memory modelers is that we tighten up the rules on how programs are allowed to pass around certain types of pointers in situations where it would be illegal to dereference them. This adds some complexity to the rules that users need to think about, but in an area (using pointers obtained via dubious means) that they already know they should tread lightly.

One thing to note is that the tradeoff I propose is pretty bad. In order to make life easier for a small number of experts (compiler writers), we're making life harder for a large number of less sophisticated users (regular C++ programmers). We should think hard about whether or not there are less extreme ways to achieve our optimization goals. (I don't have any better ideas).

§ 3.1. Background on provenance

In C and C++, pointers are not just bags of bits. If you manage to obtain some value for a Foo* with the same bytes as another Foo*, but got it through illicit means, you can't necessarily use the first pointer to access the object pointed to by the second. This lets the compiler make some useful assumptions, like:

- Pointers to locals that don't escape can be assumed not to alias other pointers.
- Accesses to an array contained in one object can be assumed not to read or write other objects (or other fields in the same object).

These in turn let the compiler generate faster or smaller code, such as by:

- Leaving values in registers, so that they can skip a store-reload sequence
- Doing common subexpression elimination (if a value can change out from under you, you can't assume it's equal to some equivalently calculated value computed elsewhere).

This should be intuitive; we need *some* way to disallow code that looks like this example from Peter Sewell.

```
#include <stdio.h>
#include <string.h>
int y=2, x=1;
int main() {
  int *p = &x + 1;
  int *q = &y;
  printf("Addresses: p=%p q=%p\n",(void*)p,(void*)q);
  if (memcmp(&p, &q, sizeof(p)) == 0) {
    *p = 11;  // does this have undefined behaviour?
    printf("x=%d y=%d *p=%d *q=%d\n",x,y,*p,*q);
  }
}
```

Provenance rules are way of saying things like "this is a disallowed buffer overrun, even if you get lucky and have the one-past-the-end-of-the-array rule let you form a valid pointer that just happens to be equal to some other valid pointer to a different object.

Unfortunately, provenance rules are very fuzzy in both C and C++. The best candidate we have for something formalized is the WG14 provenance TS.

At a high level:

- When storage is created (i.e. a stack variable, via malloc or new, etc.), it gets assigned a globally unique (through the program run) opaque integer.
- A pointer is not just an address; it's an address, together with one of those opaque integers (or a special "empty" provenance). That integer is not observable in the bytes of the pointer; it's the pointer's soul, not its body.
- Dereferencing pointers requires that the pointer's provenance matches the provenance of the underlying storage.
- There is a set of rules of provenance propagation (e.g. memcpys copy provenance), reclaiming lost provenance (e.g. ptr -> uintptr_t -> ptr), etc. These are interesting and involve complex trade-offs between compiler optimizations and ease of use, but I think we can ignore them for our purposes here.

I believe the intent for C++ is similar, except that in C++ it's objects that get provenance, not storage locations (so that, for example, the result obtained from placement-new'ing two objects at the same lo-

cation results in pointers with different provenance, even though they might have the same type and occupy the same, never-deallocated, storage).

§ 3.2. Direction for a fix

Provenance rules already have the flavor of what we want: "You have to come by your pointers honestly". We want a way to phrase the intuition that "sending a pointer value back in time" is cheating in the same way that "getting a pointer value by casting from `rand()`" is.

As a first cut, the intuition I want to capture is something like: "if some thread T1 wouldn't have the right to dereference a pointer, when it passes it to another thread T2, T2 doesn't have the right to dereference the pointer either".

One way we could do this is to add a new type of provenance. In the proposed C model, we say a pointer has either empty provenance, or some specific provenance (of some integer, say `i`); we'll call that "full provenance". We'll add something new: when we pass a pointer to another thread, it gets "provisional provenance"; provisional provenance remembers the provenance it came from, but is insufficient to grant access to the pointee until some synchronization is established (and, this applies recursively if a pointer is passed between multiple threads unsynchronized).

In pseudo-standardese ("pseudo" because the standard is not this explicit about provenance), we might say something like:

Suppose there is a pointer value V with full provenance, pointing to some object O.

If there is an atomic pointer store S of V and atomic pointer load L, where L takes its value from S, then:

- if S happens before L, L obtains V (i.e. its value has full provenance).
- If S does not happen before L, then L obtains a value V' which points to O, but with provisional provenance, contingent on L.
- On use of V', if there is some program point P such that:
    - O's lifetime start happens before P
    - L, and every load which V"s provinence is contingent upon, happens before P, and
    - P happens before the use,

then the use gets full provenance. Otherwise, it gets provisional provenance. If the use is a dereference and we have provisional provenance, then we get UB.

This is intended just to give a vibe, not to be IS-ready wording. Really we'd need to hammer down what exactly "provenance" here means, include some wording for release sequences, formalize the "sets" of pointers a little more carefully, figure out tricky interactions with things like uintptr or offsets into lazily initialized arrays, etc.

We should convince ourselves, at least informally, that this justifies our compiler rule. The reasoning is that a time-travel loop like the one we're looking at would require a load to happen before the store it got its value from; we ban this sort of thing elsewhere in the memory model, so can be fairly assured that it's unlikely.

§ 4. Some litmus tests

§ 4.1. Early escape

Recall the earlier example (the line labels have changed):

```
int* f(int* q) {
  int* p = new int;
  *p = 123;
  *q = 456; // E
  some_extern_func(*p);
  return p;
}

int dummy;
std::atomic<int *> ptr_to_int_1{&dummy};
std::atomic<int *> ptr_to_int_2{&dummy};

void t1() {
  int* q = ptr_to_int_1.load(relaxed); // D
  int* p = f(q);
  ptr_to_int_2.store(p, relaxed); // A
}

void t2() {
  int* p = ptr_to_int_2.load(relaxed); // B
  ptr_to_int_1.store(p, relaxed); // C
}
```

Here, when t2 loads its pointer value, it gets a pointer with provisional provenance (because A does not happen-before B). That pointer gets stored at C, then loaded again at D, so `q` in `t1` has provisional provenance. At `E` it is dereferenced with provisional provenance, so we're in the last set of bullet points in our provisional provenance rule. Clearly, we don't meet the conditions (only the first few lines of `f()` happen before the dereference, and the load at B doesn't happen before any of them), so the dereference acts as though we have empty provenance and we get UB.

§ 4.2. Store before fence

Copying the example again:

```
atomic<int*> atomic_int_ptr;

int* f() {
  int *p = new int;
  *p = 123; // A
  some_extern_function(); // Really, a release fence
  return p;
}

int* f() {
  int *p = new int;
  some_extern_function(); // just a release fence
  *p = 123;
  return p;
}

void message_pass_f() {
  int* p = f();
  atomic_int_ptr.store(p, relaxed);
}

void receive_message() {
  int* p = atomic_int_ptr.load(relaxed);
  atomic_thread_fence(acquire);
  if (p != nullptr) {
    assert(*p == 123);
  }
}
```

Here, when `receive_message` obtains the value `p`, it has provisional provenance. However, consider the point P immediately after the acquire fence. If `p` is non-null, then the `int`'s lifetime start happens-before P (because of the fences), the load happens before P, and P happens before the dereference. So the dereference gets full provenance, and this example doesn't have UB.

§ 4.3. Synchronized load, unsynchronized pointer

So far we've just looked at "easy" examples (where I think most people would agree there's just one obviously right answer). Here's a case where things get fuzzy.

```
std::atomic<int> flag;
std::atomic<int*> ptr_to_int;

void t1() {
    int* p = new int;
    *p = 1; // A
    flag.store(1, release); // B
    ptr_to_int.store(p, relaxed);
}

void t2() {
    if (flag.load(acquire)) {
        int* p = ptr_to_int.load(relaxed);
        if (p != nullptr) {
            // PROGRAM POINT P
            assert(*p == 1); // Must this succeed?
        }
    }
}
```

In the memory model of today, this must succeed. But the informal reasoning above makes it a little suspect. The pattern we want to allow is:

Publishing thread:

- P.1. initialize pointee
- P.2. Establish synchronization with consuming thread (maybe via a fence in some un-analyzable function)
- P.3. publish pointer

Consuming thread:

- C.1 Get pointer
- C.2 Establish synchronization for pointee
- C.3 Access pointee

In this example, though, the underlying pattern is:

- C1. Establish synchronization for pointee
- C2. Get pointer
- C3. Access pointee (no synchronization established between C2 and C3)

Nonetheless, I think the actual rule leaves this still well-defined:

- The lifetime start of the int happens-before P
- The load of the pointer happens-before P
- P happens-before the dereference

§ 4.4. Synchronized load, unsynchronized pointer, 3 threads

What if we take the previous example and move the actual dereference to a third thread?

```
std::atomic<int> flag;
std::atomic<int*> ptr_to_int_1;
std::atomic<int*> ptr_to_int_2;

void t1() {
  int* p = new int;
  *p = 1; // A
  flag.store(1, release); // B
  ptr_to_int_1.store(p, relaxed);
}

void t2() {
  int* p = ptr_to_int_1.load(relaxed);
  ptr_to_int_2.store(p, relaxed);
}

void t3() {
```

```
  if (flag.load(acquire)) {
    int* p = ptr_to_int_2.load(relaxed);
    if (p != nullptr) {
      assert(*p == 1); // Must this succeed?
    }
  }
}
```

This is an example I'm not too happy about. This program is well-defined currently, but has UB under the suggested provenance rule -- the load in `t2` is part of the set of contingent loads for the pointer with provisional provenance in `t3`. It feels very strange that relatively minor changes in where pointer copying happens can make something UB, especially in a case where there's not an underlying hardware motivation (I don't think this example is actually problematic on real hardware).

§ 5. Alternatives

§ 5.1. Do nothing, change the compilers

Another option would be to say "the standard is correct as is, and compilers ought to change". I think the rule they would have to implement is "treat any pointer that ever escapes as if it had escaped immediately upon creation". This feels heavy-handed to me; compiler-writers have put a lot of work into lifetime rules and escape analysis refinements. Contrary to popular (or at least, loudly proclaimed) belief, they mostly don't put in effort just to annoy Linus Torvalds; on average they're chasing the perf improvements, and put in effort where they hope to find them.

You can imagine some sort of annotation-based system for trying to solve this sort of perf issue. But object initialization is often a "peanut butter cost" that shows up spread around a binary. I'd be surprised if there were a hotspot users could pinpoint.

§ 5.2. Require `memory_order_load_store` for atomics

I think this solves the correctness issue while allowing aggressive compiler reordering. It has some other nice properties: it solves the out-of-thin-air problem, and localizes changes in generated code to places that use atomics.

On the other hand, there is considerable opposition from both CPU vendors and upstream compiler writers. A middle-ground might be requiring `memory_order_load_store` for atomic pointers, but not other atomics.

§ 5.3. Reverse pointer-zap, provenance stripping at store time

There are some simpler provenance-y alternatives I thought about but couldn't get workable semantics for. I'm mentioning them here in the hopes that they might inspire someone cleverer:

- Reverse pointer-zap: we could try to formalize some notion like "at the time an object's lifetime begins, all other pointers with the same value lose their provenance". (It's not clear what "at the time" here could mean; happens-before is too strong, but "anything that doesn't happen-after" is too weak).
- Provenance stripping at store time: if some thread atomically stores a pointer to some object, if the start of the object's lifetime happens-before the store, the stored pointer gets the provenance of that object; otherwise, it gets empty provenance.

The problem is that both of these directions end up breaking reasonable-seeming constructs in which pointers get passed around in a relaxed manner, only establishing synchronization in batch after some period of time has elapsed. For example, hazard pointer implementations will copy around pointers to objects they would not be allowed to access via those pointers; only the pointer value itself is used up until some synchronization point is reached. If we strip provenance without any ability to regain it, we start placing restrictions on the internal structure of those libraries.

§ 6. Questions

- Should the Early Escape example be UB?
- Should the Synchronized-load, unsynchronized pointer example be UB? (If yes, we could come up with something more aggressive).
- Where should the provenance-stripping occur? Another option is to say that it happens when some thread *stores* a pointer whose referant it cannot access.
- Where does this fit in with pointer-zap discussions more generally?

§ 7. References

Stephen Dolan's talk at FOWM discusses essentially this issue, focusing on load buffering and load/store ordering.

The "synchronized load, unsynchronized pointer" example is distilled from an LLVM issue reported by Sung-Hwan Lee. (On that thread I claimed that a provenance-based approach would render the example UB; I think I misanalyzed it and my claim there is incorrect).

The C committee has been doing a lot of work formalizing provenance rules; some of the interesting and relevant links are here, here, and here.

Hans Boehm gave a talk in Tokyo with some good examples.